

Базы данных

Лекция 10 – Berkeley DB. Утилиты

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2025

2025.12.02

Оглавление

Утилиты Berkeley DB.....	3
Переменная окружения DB_HOME.....	5
Общие параметры.....	5
Подсистема блокировки (Locking Subsystem).....	9
Подробная информация о подсистеме блокировок.....	14
Информация о блокирующих.....	18
Информация об объектах блокировки (Lock Object Information).....	21
Параметры подсистемы блокировки (Locking Subsystem Parameters).....	23
Статистика базы данных (Database Statistics).....	24
Статистика подсистемы журналирования (Logging Subsystem Statistics).....	27
Статистика кэша (Cache Statistics).....	29

Утилиты Berkeley DB

Berkeley DB позволяет реализовывать административные задачи, такие как резервное копирование, контрольные точки и архивирование журналов в рамках приложения.

После реализации этих задач и правильной настройки базы данных, она не потребует внешнего администрирования и настройки. Однако на этапе разработки понадобятся инструменты для устранения неполадок и настройки, которые не входят в состав приложения.

Berkeley DB поставляется с несколькими полезными утилитами, которые для этой цели можно использовать. Хорошее понимание этих утилит и предоставляемой ими информации может значительно сократить время, затрачиваемое на устранение неполадок в приложении на базе Berkeley DB.

Утилиты Berkeley DB предоставляют возможность выполнять административные задачи в базе данных без необходимости написания кода.

Почти все функции, выполняемые этими утилитами, можно реализовать программно, однако часто возникает необходимость быстро протестировать что-то и не хочется тратить часы на написание одноразового кода. Для этой цели в большинстве случаев следует использовать утилиты Berkeley DB. При разработке приложения одни утилиты будут использоваться чаще других.

В таблице на следующем слайде утилиты перечислены в порядке их полезности.

В зависимости от требований приложения, понимание наиболее полезных утилит может отличаться от этого порядка.

Большинство приложений обычно выполняют ряд перечисленных функций программно, поэтому можно обнаружить, что некоторые из перечисленных утилит никогда не пригодятся. Тем не менее, всё равно следует ознакомиться с информацией, предоставляемой этими утилитами, поскольку иногда случается, когда мы ищем определённую информацию для отладки приложения, мы можем и не знать, что её можно легко получить с помощью одной из утилит.

Утилита	Описание
db_stat	Отображает статистику различных подсистем в среде Berkeley DB.
db_recover	Восстанавливает среду Berkeley DB.
db_dump	Создаёт дамп содержимого базы данных.
db_load	Загружает данные в базу данных.
db_checkpoint	Создаёт контрольную точку базы данных.
db_deadlock	Выполняет обнаружение взаимоблокировок в среде.
db_printlog	Печатает содержимое журнала транзакций.
db_archive	Выполняет функции архивирования базы данных.
db_hotbackup	Выполняет горячее резервное копирование файлов данных и журналов транзакций.
db_verify	Проверяет целостность базы данных.

Переменная окружения DB_HOME

Все утилиты Berkeley DB чувствительны к переменной окружения DB_HOME.

Если переменная DB_HOME установлена в оболочке, из которой запущена утилита, то для выполнения утилиты будет использоваться окружение Berkeley DB, находящаяся в каталоге, на который указывает переменная окружения. Если переменная DB_HOME не установлена, то для выполнения будет использоваться текущий рабочий каталог.

■ Если приложение работает с несколькими окружениями, при использовании утилит лучше не устанавливать переменную DB_HOME. Таким образом, можно избежать ситуаций, когда можно было бы ожидать, что утилита будет работать с текущим рабочим каталогом, но на самом деле она работает с каталогом, указанным в переменной DB_HOME.

Общие параметры

Следующие параметры поддерживаются практически всеми утилитами и выполняют одну и ту же функцию везде, где они поддерживаются.

-h

Используется для указания домашнего каталога окружения.

Даже если задан параметр DB_HOME, можно использовать опцию -h для его переопределения. Если не заданы ни DB_HOME, ни -h, будет использоваться окружение, находящееся в текущем рабочем каталоге.

-v

Используется для запуска утилиты в режиме расширенного вывода. Подробные сообщения о выполнении при указании этой опции выводятся на стандартный вывод.

Эта опция не поддерживается db_stat, db_printlog, db_load и db_dump, поскольку эти утилиты уже по умолчанию возвращают подробную информацию.

-P

При использовании ENV->set_encrypt() для включения шифрования в окружении, можно использовать опцию -P для указания пароля, который утилита будет использовать для расшифровки файлов окружения. Пароль должен совпадать с паролем, установленным с помощью ENV->set_encrypt().

-N

При указании этой опции утилита не пытается захватывать мьютексы при обходе структур данных окружения.

Этот инструмент полезен для отладки повреждённых окружений. Если процесс аварийно завершается, удерживая мьютекс в критической секции общей памяти окружения, его нельзя использовать до тех пор, пока не будет выполнено восстановление.

Запуск восстановления фактически уничтожает существующее окружение и создаёт новое, тем самым уничтожая все признаки проблемы. С помощью этой опции можно осмотреть окружение, не разрушая его.

-V

Все утилиты Berkeley DB поддерживают опцию -V.

При запуске с этой опцией каждая утилита выводит номер версии библиотеки Berkeley DB, с которой она скомпонована, и затем завершает работу.

Независимо от того, какая утилита будет запущена с опцией -V, буде получен одинаковый вывод, типа этого:

```
$ db_stat -V
Sleepycat Software: Berkeley DB 4.5.20: (September 20, 2006)
$ db_printlog -V
Sleepycat Software: Berkeley DB 4.5.20: (September 20, 2006)
```

При работе с несколькими версиями библиотеки эта опция может быть чрезвычайно полезна. Если версия библиотеки, использованная для создания среды Berkeley DB, не совпадает с версией библиотеки, связанной с утилитой, можно увидеть странные ошибки, типа:

```
$ db_stat -CA
db_stat: DB_ENV->open: Invalid argument
```

Вывод `db_stat`, показанный здесь, является результатом запуска `db_stat`, скомпилированной с версией 4.4.20, в среде, созданной в версии 4.3.28. При возникновении таких ошибок первым делом следует проверить версию библиотеки. Начиная с версии 4.5.20, все утилиты Berkeley DB могут обнаруживать несоответствие версий и сообщать об ошибке.

■ **Примечание.** При запуске с опцией `-V` утилиты Berkeley DB сообщают номер версии библиотеки, на которую они ссылаются, не открывая среду базы данных. Поэтому они не завершают работу из-за несоответствия версий.

db_stat

`db_stat` — самая важная утилита, предоставляемая Berkeley DB. Она также широко считается самой запутанной. Она сообщает много ценной информации, но для понимания вывода требуются практика.

`db_stat` сообщает различные типы статистики окружения базы данных, отсюда и название `db_stat`.

Окружение базы данных состоит из различных подсистем. Каждая подсистема поддерживает множество статистических параметров, относящихся к её использованию.

`db_stat` предоставляет доступ к этим параметрам через различные параметры командной строки. В таблице ниже представлены основные опции утилиты.

Опция	Подсистема
-C	Подробная внутренняя информация о подсистеме блокировки; полезно для отладки.
-c	Статистика подсистемы блокировки.
-l	Статистика подсистемы журналирования.
-M	Подробная внутренняя информация о пуле памяти базы данных или кэше.
-m	Статистика кэша базы данных.
-R	Подробная внутренняя информация о подсистеме репликации.
-r	Быстрая статистика подсистемы репликации.
-d	Статистика для конкретного файла базы данных. Если файл содержит несколько баз данных, эта опция выведет статистику для базы данных, содержащей информацию о базах данных, хранящихся в файле.
-s	Используется вместе с опцией -d. Если файл содержит несколько баз данных, то имя файла будет указано с помощью опции -d, а имя базы данных — с помощью опции -s.
-t	Отображает информацию о транзакционной подсистеме.
-e	Статистика, относящаяся ко всей среде; совокупность статистики всех подсистем.

Подсистема блокировки (Locking Subsystem)

Параметры -С и -с отображают информацию о подсистеме блокировки. Параметр -с предоставляет сводку основных параметров блокировки, не вдаваясь в подробности.

Эта информация полезна в двух случаях: для определения численных параметров подсистемы блокировки и для измерения ее производительности.

```
$ db_stat -c
34      Last allocated locker ID
0x7fffffff Current maximum unused locker ID
9       Number of lock modes
1000    Maximum number of locks possible
1000    Maximum number of lockers possible
1000    Maximum number of lock objects possible
5       Number of current locks
9       Maximum number of locks at any one time
14      Number of current lockers
17      Maximum number of lockers at any one time
5       Number of current lock objects
8       Maximum number of lock objects at any one time
166     Total number of locks requested
161     Total number of locks released
0       Total number of locks upgraded
7       Total number of locks downgraded
1       Lock requests not available due to conflicts, for which we waited
0       Lock requests not available due to conflicts, for which we did not wait
0       Number of deadlocks
0       Lock timeout value
```

0	Number of locks that have timed out
0	Transaction timeout value
0	Number of transactions that have timed out
344KB	The size of the lock region
0	The number of region locks that required waiting (0%)

Определение размеров подсистемы блокировок

Подсистема блокировок хранит информацию о блокировках в таблицах блокировок фиксированного размера, расположенных в области общей памяти среды базы данных.

Вывод команды `db_stat -c` позволяет определить размер таблицы блокировок. Следующие значения относятся к информации о размере:

Maximum number of locks possible — максимально возможное количество блокировок.

Показывает максимальное количество записей блокировок, которые в данный момент могут быть одновременно сохранены в таблице блокировок.

Значение по умолчанию — 1000. Можно изменить его, вызвав `set_lk_max_lockers()` перед вызовом `open()` или добавив строку `set_lk_max_locker <требуемое число>` в файл `DB_CONFIG`.

Maximum number of lockers possible — максимально возможное количество блокирующих.

Показывает максимальное количество блокирующих, которое может быть выделено в таблице блокировок в любой момент времени.

Значение по умолчанию — 1000, его можно изменить, вызвав `set_lk_max_locks()` перед вызовом `open()` или добавив `set_lk_max_locks <нужное число>` в файл `DB_CONFIG`.

Maximum number of lock objects possible — максимально возможное количество заблокированных объектов.

Показывает максимальное количество объектов, которые могут быть заблокированы в системе одновременно. Значение по умолчанию также равно 1000.

Можно изменить его, вызвав `set_lk_max_objects()` перед вызовом `open()` или указав `set_lk_max_objects <требуемое значение>` в файле `DB_CONFIG`.

The size of the lock region — размер области блокировки.

Показывает размер, выделенный в области общей памяти для таблицы блокировок.

Lock timeout value — указывает максимальную продолжительность (при условии, что обнаружение взаимоблокировок выполняется достаточно часто), в течение которой блокировка будет храниться в таблице блокировок.

При определённых обстоятельствах возможно, что большее время ожидания может привести к увеличению количества записей в таблице блокировок, поскольку блокировки будут дольше выходить из строя и дольше оставаться в таблице блокировок.

Самый простой способ определить оптимальные значения этих параметров — запустить приложение при максимальной нагрузке, которой может подвергнуться система, а затем выполнить команду `db_stat -s`, чтобы увидеть фактическое количество блокировок, блокировщиков и объектов блокировок, используемых системой.

Затем следует установить максимальные ограничения в два раза больше фактических значений. Таким образом, можно быть уверенным, что ограничение не будет превышено ни для одного из этих параметров даже в условиях нагрузки.

Когда в системе заканчивается место в таблице блокировок, будет выдана ошибка `ENOMEM` для всех операций с базой данных, требующих блокировки.

Если библиотека скомпилирована с флагом — `enable-debug`, то библиотека также выведет более подробное диагностическое сообщение, указывающее, какой из этих трёх параметров превысил ограничение.

Если вызов Berkeley DB возвращает ошибку `ENOMEM`, это значит, что пора выполнить команду `db_stat -c`.

■ API Berkeley DB возвращают ошибку `ENOMEM`, когда системе не хватает ресурсов (например, памяти или дискового пространства), или когда внутренние структуры данных, используемые для работы базы данных, не имеют правильного размера.

Одной из таких структур данных является таблица блокировок. Информацию о её размере можно получить, выполнив команду `db_stat -c`.

Измерение производительности подсистемы блокировки

Можно оценить производительность подсистемы блокировки, посмотрев на следующие значения, возвращаемые командой `db_stat -c`:

Maximum number of locks at any one time — максимальное количество блокировок одновременно.

Это максимальное количество одновременных блокировок, выделяемое библиотекой. Если это количество превышает половину максимального количества возможных блокировок, следует увеличить максимально возможное количество блокировок, поскольку существует вероятность скорого превышения лимита. Аналогичным образом, следует проверить значения максимального количества блокировок одновременно и максимального количества объектов блокировки одновременно, чтобы убедиться, что размер таблицы блокировок выбран правильно.

Number of deadlocks — количество взаимоблокировок.

Показывает количество запросов на блокировку, приведших к взаимоблокировкам.

Когда библиотека определяет, что запрос на блокировку может привести к взаимоблокировке, она немедленно освобождает один из блокировщиков, не дожидаясь истечения времени ожидания его блокировки.

Слишком большое количество взаимоблокировок указывает либо на проблемы в логике приложения, которые приводят к чрезмерному количеству конфликтов, либо на неправильный размер страницы базы данных, что может приводить к блокировке слишком большого количества страниц во время операций с базой данных.

Number of locks that have timed out — количество блокировок, истёкших по времени.

Показывает количество блокировок, истёкших по времени.

Если сообщается о слишком большом количестве тайм-аутов, вероятно, следует увеличить значение тайм-аута блокировки, поскольку блокировщикам не хватает времени для выполнения своей работы.

Большинство других значений, возвращаемых командой `db_stat -c`, используются лишь изредка, но о них полезно знать на случай, если придется столкнуться со сложной проблемой, требующей дополнительной информации.

Подробная информация о подсистеме блокировок

Опция -С. Вывод этой опции показывает содержимое таблицы блокировок. Опция полезна для отладки утечек (полученных, но не снятых) блокировок, потерянных (не зафиксированных и не прерванных) транзакций и утечек (открытых, но не закрытых) дескрипторов базы данных.

Чтобы полностью использовать эту информацию, следует получить идентификаторы транзакций и блокировок из кода приложения.

Опция -С имеет ряд подопций:

Имя субопции	Описание
А	Отображает информацию по всем подпараметрам
с	Показывает матрицу конфликтов блокировок
l	Отображает информацию о блокировках
о	Отображает информацию об объектах блокировки
р	Отображает параметры подсистемы блокировки

Матрица конфликтов блокировок

Можно посмотреть матрицу конфликтов блокировок, выполнив команду `db_stat -Cc`:

```
$ db_stat -Cc
=====
Lock REGINFO information:
Lock      Region type
5         Region ID
__db.005   Region name
0x4030e000 Original region address
0x4030e000 Region address
0x40363f40 Region primary address
0         Region maximum allocation
0         Region allocated
REGION_JOIN_OK Region flags
=====
Lock conflict matrix:
0  0  0  0  0  0  0  0  0
0  0  1  0  1  0  1  0  1
0  1  1  1  1  1  1  1  1
0  0  0  0  0  0  0  0  0
0  1  1  0  0  0  0  1  1
0  0  1  0  0  0  0  0  1
0  1  1  0  0  0  0  1  1
0  0  1  0  1  0  1  0  0
0  1  1  0  1  1  1  0  1
```

В первом разделе можно увидеть некоторую информацию об общей области памяти, где хранится таблица блокировок. Эта информация не особо полезна, если используется только Berkeley DB, но может быть полезна при устранении неполадок в библиотеке.

Вторая часть вывода показывает матрицу конфликтов блокировок, которую библиотека использует для разрешения взаимоблокировок. Строки матрицы представляют собой удерживаемые блокировки, а столбцы — запрошенные.

Цифра 1 означает, что запрошенная блокировка не может быть предоставлена, а 0 — что она может быть предоставлена.

Показанная здесь матрица блокировок — это фактическая матрица, используемая библиотекой. Она немного отличается от той, которая доступна (для изменения) через API блокировок `set_lk_conflicts()`.

В API доступно только шесть режимов блокировок, тогда как здесь доступно девять.

Если изучить исходный код, чтобы узнать режимы блокирования (lock-node), можно увидеть матрицу, которая выглядит следующим образом:

	N	R	W	WT	IW	IR	RIW	DR	WW
N	0	0	0	0	0	0	0	0	0
R	0	0	1	0	1	0	1	0	1
W	0	1	1	1	1	1	1	1	1
WT	0	0	0	0	0	0	0	0	0
IW	0	1	1	0	0	0	0	1	1
IR	0	0	1	0	0	0	0	0	1
RIW	0	1	1	0	0	0	0	1	1
DR	0	0	1	0	1	0	1	0	0
WW	0	1	1	0	1	1	1	0	1

Аббревиатуры режима блокирования

Lock-Mode	Описание
N	Блокировка не предоставлена
R	Блокировка чтения
W	Блокировка записи
WT	Ожидание блокировки
IW	Намерение записи
IR	Намерение чтения
RIW	Чтение с намерением записи (RMS/RMV)
DR	Некорректное чтение
WW	Была запись

Berkeley DB позволяет изменять матрицу блокировок с помощью API блокировки.

Однако это нежелательно и оправдано только в том случае, когда реализуется собственная база данных или реализуется фреймворк блокировок с использованием API блокировки Berkeley DB.

Информация о блокирующих

Текущие записи в таблице блокирующих отображаются при запуске **db_stat -Cl**:

```
$ db_stat -Cl
=====
Lock REGINFO information:
Lock   Region type
5      Region ID
__db.005 Region name
0x4030e000 Original region address
0x4030e000 Region address
0x40363f40 Region primary address
0        Region maximum allocation
0        Region allocated
REGION_JOIN_OK Region flags
=====
Locks grouped by lockers:
Locker  Mode  Count Status ----- Object -----
  10 dd=15 locks held 1 write locks 0 pid/thread 9738/1078930528
  10 READ 1 HELD Person_Db handle 0
  11 dd=14 locks held 0 write locks 0 pid/thread 9738/1078930528
  12 dd=13 locks held 1 write locks 0 pid/thread 9738/1078930528
  12 READ 1 HELD Account_Db handle 0
  13 dd=12 locks held 0 write locks 0 pid/thread 9738/1078930528
  14 dd=11 locks held 1 write locks 0 pid/thread 9738/1078930528
  14 READ 1 HELD Bank_Db handle 0
  15 dd=10 locks held 0 write locks 0 pid/thread 9738/1078930528
```

```

16 dd= 9 locks held 1 write locks      0 pid/thread 9738/1078930528
16 READ 1 HELD Person_Name_Db handle 0
18 dd= 8 locks held 1 write locks      0 pid/thread 9738/1078930528
18 READ 1 HELD Person_Dob_Db handle 0
1a dd= 6 locks held 0 write locks      0 pid/thread 9738/1082035120
1b dd= 5 locks held 0 write locks      0 pid/thread 9738/1082035120
1c dd= 4 locks held 0 write locks      0 pid/thread 9738/1082035120
1d dd= 3 locks held 0 write locks      0 pid/thread 9738/1082035120
1e dd= 2 locks held 0 write locks      0 pid/thread 9738/1082035120
1f dd= 1 locks held 0 write locks      0 pid/thread 9738/1082035120
8000000b dd= 0 locks held 4 write locks 3 pid/thread 19076/1082047408
8000000b WRITE 2 HELD Person_Db        page 1
8000000b WRITE 1 HELD Person_Name_Db   page 1
8000000b WRITE 4 HELD Person_Dob_Db    page 1
8000000b READ 1 HELD Person_Db         page 1

```

В начале отображается информация о блокировке региона, которая идентична той, что отображалась при запуске `db_stat -Cc`.

За информацией о регионе следует список блокировок, присутствующих в данный момент в таблице блокировок. Список содержит информацию о блокировке и заблокированных ею объектах. Например, блокировка с идентификатором 10 принадлежит потоку 1078930528 в процессе 9738 и в настоящее время удерживает блокировку чтения (READ) на дескрипторе базы данных `Person_Db`.

Также видно, что существует ряд блокировок (1a, 1b, 1c и т. д.), которые не удерживают блокировок. Эти записи блокировок в конечном итоге будут повторно использованы при выделении новых блокировок.

■ Примечание. Информация об идентификаторах процесса и потока будет доступна в `db_stat -Cl` только в том случае, если для установки идентификаторов использовался метод `set_thread_id()`.

Идентификаторы блокировщиков (локеров) с большими значениями принадлежат транзакциям.

В выходных данных можно видеть, что идентификатор блокировщика `8000000b` принадлежит транзакции. Эта транзакция удерживает четыре блокировки, из которых три являются блокировками записи (`WRITE`).

Сведения о блокировках указывают на то, что блокировки, удерживаемые транзакцией, являются страничными, а не дескрипторными.

В выходных данных следует обратить внимание на поле `dd`, которое указывает идентификатор детектора взаимоблокировок, назначенный блокировщику. Можно заметить, что иногда при запуске `db_stat` поле `dd` равно 0 для всех блокировщиков.

Это происходит, когда в системе не обнаружено взаимоблокировок.

При обнаружении первой взаимоблокировки блокировщикам присваиваются идентификаторы.

Для блокировщика `8000000b` значение `dd` равно 0, поскольку он мог быть запущен после того, как детектор взаимоблокировок назначил идентификаторы.

Информация об объектах блокировки (Lock Object Information)

Информацию об объектах блокировки можно получить, выполнив `db_stat -Co`. Она предоставляет практически ту же информацию, что и `db_stat -Cl`, но в другом формате:

```
$ db_stat -Co
=====
Lock REGINFO information:
Lock Region type
5      Region ID
__db.005      Region name
0x4030e000    Original region address
0x4030e000    Region address
0x40363f40    Region primary address
0            Region maximum allocation
0            Region allocated
REGION_JOIN_OK Region flags
=====
Locks grouped by object:
Locker      Mode      Count Status      Object -----
8000000b    READ      1 HELD      Person_Db    page          1
8000000b    WRITE     2 HELD      Person_Db    page          1
          10    READ      1 HELD      Person_Db    handle
          12    READ      1 HELD      Account_Db   handle
          14    READ      1 HELD      Bank_Db      handle
8000000b    WRITE
          16    READ      1 HELD      Person_Dob_Db page
          16    READ      1 HELD      Person_Name_Db handle
8000000b    WRITE      Person_Name_Db page
```

18	READ	1 HELD	Person_Dob_Db	handle
----	------	--------	---------------	--------

Как и раньше, сначала отображается информация о зоне блокировки, а затем — сами блокировки.

Разница заключается в том, что здесь блокировки отсортированы по объектам блокировки, а не по самим блокировкам.

Параметры подсистемы блокировки (Locking Subsystem Parameters)

Выполнив команду `db_stat -Cp`, можно получить информацию о параметрах подсистемы блокировки:

```
$ db_stat -Cp
Lock REGINFO information:
Lock Region type
5      Region ID
__db.005      Region name
0x4030e000      Original region address
0x4030e000      Region address
0x40363f40      Region primary address
0      Region maximum allocation
0      Region allocated
REGION_JOIN_OK Region flags
Lock region parameters:
46      Lock region region mutex [1/269 0% 21588/1076295360]
1031      locker table size
1031      object table size
343720 obj_off
335464 locker_off
0      need_dd
```

Важная информация в этом выводе — мьютекс области блокировки.

Он показывает уровень конкуренции за мьютекс области блокировки.

Если это значение увеличивается, это означает, что слишком много потоков пытаются использовать базу данных слишком долго.

Статистика базы данных (Database Statistics)

Можно получить доступ к статистике базы данных, выполнив команду `db_stat -d`.

Информация, возвращаемая этой командой, весьма полезна для настройки метода доступа к базе данных. Эта команда выводит основную информацию о базе данных, включая время создания, метод доступа, порядок байтов, размер страницы и т. д. Она также выводит важную информацию о настройке.

```
$ db_stat -d Person_Db
Sun Jun 11 17:55:32 2006    Local time
53162    Btree magic number
9        Btree version number
Little-endian    Byte order
Flags
2        Minimum keys per-page
512     Underlying database page size
1        Number of levels in the tree
5        Number of unique keys in the tree
5        Number of data items in the tree
0        Number of tree internal pages
0        Number of bytes free in tree internal pages (0% ff)
1        Number of tree leaf pages
250    Number of bytes free in tree leaf pages (51% ff)
0        Number of tree duplicate pages
0        Number of bytes free in tree duplicate pages (0% ff)
0        Number of tree overflow pages
0        Number of bytes free in tree overflow pages (0% ff)
0        Number of empty pages
```


Размер страницы базы данных (Database Page Size)

Размер страницы базы данных определяется по параметру **Underlying database page size**.

Это минимальный объём данных, блокируемый одной блокировкой. Также определяет квант времени, за который данные загружаются на диск и считываются с него.

Если размер страницы слишком мал, то будет выполняться слишком много дисковых операций ввода-вывода, что замедлит работу приложения.

Если размер страницы слишком велик, приложение потеряет параллелизм, поскольку каждая блокировка будет блокировать слишком много данных.

Чтобы оценить, не слишком ли мал размер страницы, следует проверить коэффициент попадания в кэш (сообщаемый командой `db_stat -m`).

Если размер страницы слишком мал, то есть если большинство записей не помещаются на одной странице, коэффициент попадания в кэш будет ниже (менее 80%).

Чем меньше размер страницы, тем больше будет создаваться записей переполнения, что приведёт к снижению коэффициента попадания в кэш.

Определить, не слишком ли велик размер страницы, можно по количеству взаимоблокировок (сообщаемому командой `db_stat -c`).

Если слишком много взаимоблокировок, которые невозможно объяснить степенью параллелизма в приложении, то размер страницы базы данных, вероятно, слишком велик.

При определении правильного размера страницы следует учитывать количество страниц переполнения. Страница переполнения создаётся, когда запись не помещается полностью на странице. Они значительно снижают производительность базы данных, поскольку некоторым записям может потребоваться несколько операций ввода-вывода на диск. Если слишком много страниц переполнения, следует рассмотреть возможность увеличения размера страницы базы данных.

Коэффициент заполнения страницы (Page-Fill Factor)

Коэффициент заполнения страницы — это средний процент используемого пространства на страницах базы данных.

Если он слишком низок, страницы базы данных будут заполнены слишком слабо, что сделает блокировку и подкачку на диск и с него неэффективными.

Меньший коэффициент заполнения снижает эффективность использования кэша.

Поскольку каждая страница содержит меньше записей, вероятность того, что искомая запись находится в кэше, меньше.

Коэффициент заполнения определяется числом свободных байтов на страницах листьев дерева (процент указан в скобках).

Коэффициент заполнения менее 80% считается плохим. В показанном ранее дампе коэффициент заполнения составляет всего 51%. Он настолько низок, поскольку в базе данных всего пять записей, занимающих лишь половину страницы.

На коэффициент заполнения влияют `DB->set_bt_minkey( )` для метода доступа BTree и `DB->set_h_ffactor( )` для метода доступа Hash.

Увеличивая минимальное количество ключей, которые должны присутствовать на одной странице, можно увеличить коэффициент заполнения.

На коэффициент заполнения также влияет флаг `DB_REVSPLITOFF` в методе `DB->set_flags()`. Если флаг установлен `DB_REVSPLITOFF`, страницы не будут сжиматься при удалении записей из базы данных. При регулярном добавлении и удалении записей в базе данных этот флаг предотвратит чрезмерные взаимоблокировки, но также снизит коэффициент заполнения.

Статистика подсистемы журналирования (Logging Subsystem Statistics)

Статистика подсистемы журналирования выводится при выполнении `db_stat -l`:

```
$ db_stat -l
0x40988 Log magic number
11      Log version number
32KB    Log record cache size
0       Log file mode
10Mb    Current log file size
57      Records entered into the log
3KB 683B Log bytes written
0       Log bytes written since last checkpoint
18      Total log file I/O writes
0       Total log file I/O writes due to overflow
18      Total log file flushes
227     Total log file I/O reads
1       Current log file number
1146448 Current log file offset
1       On-disk log file number
1146448 On-disk log file offset
1       Maximum commits in a log flush
0       Minimum commits in a log flush
96KB    Log region size
0       The number of region locks that required waiting (0%)
```

Эта команда выводит некоторую стандартную информацию о журналах транзакций, включая номер текущего файла журнала, текущее смещение файла журнала и общее количество байтов журнала, записанных на диск. Здесь содержится мало информации, которую можно использовать для настройки системы. Единственное значение, которое можно использовать для настройки, — это размер кэша записей журнала.

Кэш журнала транзакций (Transaction Log Cache)

Кэш журнала используется для хранения журналов транзакций перед их сбросом в постоянное хранилище.

При настройке ведения журнала в оперативной памяти размер самой большой транзакции, которую фиксирует приложение, всегда должен быть меньше размера кэша журнала.

По умолчанию размер этого кэша для ведения журнала в оперативной памяти составляет 1 МБ. При попытке зафиксировать транзакцию размером более 1 МБ приложение зависнет или аварийно завершит работу.

Кэш журнала не может быть выгружен на страницу в конфигурации с ведением журнала в оперативной памяти, поэтому, если какая-либо транзакция не помещается в кэш, база данных мало что может с этим поделать.

Если имеют место необъяснимые зависания или сбои в приложении, следует обязательно проверить размер кэша журнала — возможно, некоторые транзакции переполняют кэш.

При настройке ведения журнала на диск размер кэша по умолчанию составляет 32 КБ. В дисковом режиме кэш сбрасывается на диск при его заполнении, например, при выполнении длительной или большой транзакции. Увеличение размера кэша при необходимости выполнения больших или длительных транзакций повышает производительность базы данных.

Размер журналов транзакций (Size of Transaction Logs)

Можно узнать точный размер журналов транзакций, используя Current log file number и Current log file size.

Если размер файла журнала слишком велик, то, просто глядя на имена файлов журнала транзакций, невозможно определить их размер.

Также можно узнать, сколько журналов было записано на диск, посмотрев на On-disk log file number и On-disk log file offset.

Статистика кэша (Cache Statistics)

Статистика кэша отображается командой db_stat -m. В первом разделе отображается общая статистика кэша, а затем статистика отдельных кэшей базы данных.

Эта команда выводит некоторые важные настройки, а также общую информацию о системе. Одним из важнейших параметров настройки является размер кэша. Этот кэш используется для хранения страниц базы данных в области общей памяти среды.

```
$ db_stat -m
257KB 868B      Total cache size
1      Number of caches
264KB   Pool individual cache size
0      Maximum memory-mapped file size
0      Maximum open file descriptors
0      Maximum sequential buffer writes
0      Sleep after writing maximum sequential buffers
0      Requested pages mapped into the process' address space
163     Requested pages found in the cache (95%)
```

8	Requested pages not found in the cache
0	Pages created in the cache
8	Pages read into the cache
4	Pages written from the cache to the backing file
0	Clean pages forced from the cache
0	Dirty pages forced from the cache
0	Dirty pages written by trickle-sync thread
8	Current total page count
8	Current clean page count
0	Current dirty page count
37	Number of hash buckets used for page location
179	Total number of times hash chains searched for a page
1	The longest hash chain searched for a page
163	Total number of hash chain entries checked for page
0	The number of hash bucket locks that required waiting (0%)
0	The maximum number of times any hash bucket lock was waited for
0	The number of region locks that required waiting (0%)
28	The number of page allocations
0	The number of hash buckets examined during allocations
0	The maximum number of hash buckets examined for an allocation
0	The number of pages examined during allocations
0	The max number of pages examined for an allocation
Pool File: Person_Db	
512	Page size
0	Requested pages mapped into the process' address space
86	Requested pages found in the cache (97%)
2	Requested pages not found in the cache
0	Pages created in the cache

2	Pages read into the cache
2	Pages written from the cache to the backing file

Размер кэша (Cache Size)

Размер кэша базы данных — это самый большой регулятор. Можно значительно повысить производительность системы, правильно установив размер кэша.

По умолчанию он равен 256 КБ. Чаще всего этого слишком мало. Чем больше размер кэша, тем меньше дисковых операций ввода-вывода потребуется для обслуживания запросов к базе данных.

Такой малый размер кэша по умолчанию обусловлен историческими причинами.

Berkeley DB изначально была написана в 1991 году, когда память была довольно дорогой. При 64 КБ (первоначальном значении по умолчанию) в кэше можно было разместить 128 страниц по 512 байт, и это было много в то время. Первоначально значение по умолчанию было установлено на низком уровне, чтобы не допустить чрезмерного использования ресурсов каким-либо одним приложением на часто общих исследовательских машинах.

Коэффициент попадания в кэш (Cache Hit Ratio)

Коэффициент попадания в кэш определяется как процент запросов, обслуженных кэшем. Чем выше коэффициент попадания, тем меньше дисковых операций ввода-вывода требуется для обслуживания запросов к базе данных. Коэффициент попадания в кэш отображается в разделе Requested pages found in the cache.

Если это значение мало (менее 90%), следует рассмотреть возможность увеличения размера кэша.